

# 1. You probably want the other example

If you have arrived here looking for math support, please read the `hello` example first. The bundled Noto Sans family now resolves math operators ( $\leq \geq \neq \sqrt{\partial} \sum -$ ) automatically through per-glyph fallback to the embedded Noto Sans Math cut, mid-line, without `#set text(font: ...)` ceremony. That is the path you want for almost every document.

This example exists for the *other* case: when you, for some defensible reason, want to **avoid** embedding font data and let the reader's built-in Helvetica supply the outlines.

`#set text(font: "Helvetica")` selects that path. The PDF ships without `/FontFile2` streams, file size drops by roughly 340 KB, and the reader is asked nicely to render ten specific math glyphs from its own AGL-named Core 14 metrics.

**Asked nicely** is doing a lot of work in that sentence.

## 1.1. What you get if you stay on this path

The Adobe Core 14 AFMs name ten math glyphs, accessible through a per-document `/Differences` array layered on top of `WinAnsiEncoding`:

1. `lessequal` ( $\leq$ ),
2. `greaterequal` ( $\geq$ ),
3. `notequal` ( $\neq$ ),
4. `partialdiff` ( $\partial$ ),
5. `summation` ( $\sum$ ),
6. `radical` ( $\sqrt{\phantom{x}}$ ),
7. `minus` ( $-$ , the real one, U+2212, *not* the ASCII hyphen-minus on your keyboard),
8. `Delta` ( $\Delta$ , the math increment, U+2206; the Greek capital letter delta at U+0394 is a different codepoint and is *not* reachable on this path),
9. `lozenge` ( $\diamond$ ),
10. `fraction` ( $\frac{\phantom{x}}{\phantom{x}}$ , U+2044, *not* the ASCII slash).

A worked sample, for the snapshot:

Inequalities:  $a \leq b$ ,  $x \geq y$ ,  $p \neq q$ . Calculus:  $\partial f / \partial x = 0$ ,  $\sum a = 1$ ,  $\sqrt{n} \geq 0$ . Geometry:  $\triangle ABC$ , optional steps marked  $\diamond$ . The minus operator ( $a - b$ ) is U+2212, distinct from the hyphen ( $a - b$ ); the fraction slash ( $1/2$ ) is U+2044, distinct from `/`.

## 1.2. What you do not get

Everything else. Off the top, and in no particular order: the infinity symbol, the universal and existential quantifiers, the element-of relation, nabla, the product operator, the circled operators (`oplus`, `otimes`, and their siblings), sub- and superscripts, the calligraphic and blackletter alphabets every paper on algebra leans on, and most Greek letters used as math variables.

We have deliberately not pasted those codepoints into this paragraph, because the Base14 path substitutes `?` for anything outside `WinAnsiEncoding` and the AGL math subset, and a wall of `?` is not the demo we are going for here. If you would like to see the substitution live, type one in: `?` becomes `?`, `?` becomes `?`, the Greek letter you wanted becomes `?`. There is no `/Differences` trick that conjures glyphs out of a font that does not contain them. For any

math beyond the ten glyphs above, switch back to the embedded fallback path.

## 1.3. The substitution problem

Even within the ten supported glyphs, what the reader actually paints depends on which font fontconfig hands the reader as its "Helvetica." Different installs of the same reader can end up with different substitutes:

- Lozenge (◊) is the most fragile: many free Helvetica substitutes simply do not contain it, and you get an empty box.
- Adobe Reader has been observed splicing in spacing-substitution composition for /Differences-mapped glyphs, most visibly for the Romanian comma-below characters ț and ș.
- Within the same Poppler-based reader family (GNOME Papers, Evince, zathura), different installs can pick different substitutes via fontconfig and end up with different glyph coverage.

If you target a specific reader and rely on any of these glyphs, render on that reader before you ship. We will not do this for you. We barely do it for ourselves.